

Project Design Phase Solution Architecture

Date	12 July 2026
Team ID	
Project Name	DHARSHANEASE
Maximum Marks	4 Marks

Solution Architecture:

DarshanEase follows a **three-tier MERN architecture** consisting of a Presentation Layer, Application Layer, and Database Layer. The system enables devotees to register, log in, browse temples, view darshan slot availability, book tickets, make donations, and manage their bookings. Administrators can manage temples, slots, bookings, donations, and temple images through an integrated dashboard.

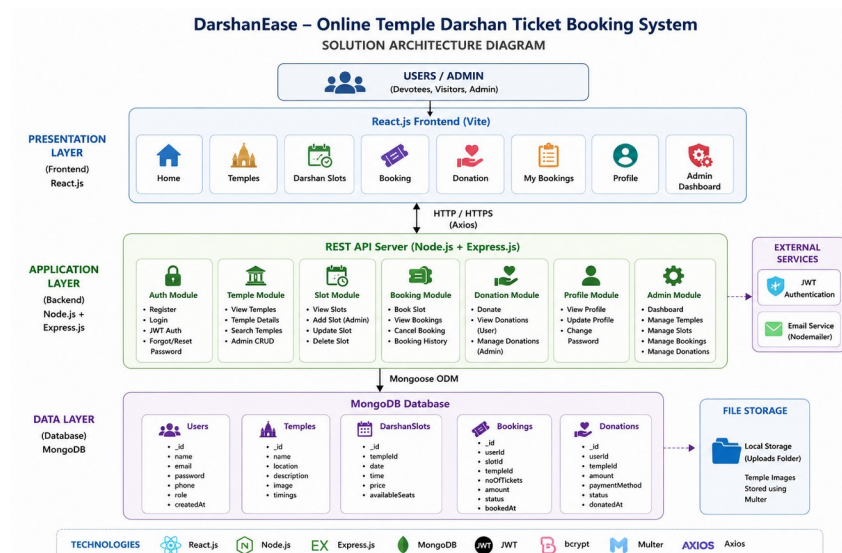
The **Presentation Layer** is developed using **React.js** with React Router for navigation and Axios for communication with the backend APIs.

The **Application Layer** is developed using **Node.js** and **Express.js**, where REST APIs handle user authentication, temple management, slot management, booking operations, donation management, and profile management. Passwords are encrypted using **bcrypt**, while **JWT** is used for secure authentication and authorization.

The **Data Layer** uses **MongoDB** with **Mongoose** to store Users, Temples, Darshan Slots, Bookings, and Donations. Temple images are uploaded using **Multer** and stored in the local uploads directory.

This layered architecture separates presentation, business logic, and data storage, making the application modular, scalable, maintainable, and easy to extend.

Solution Architecture Diagram:



Data Flow

1. User accesses the React web application.
2. User registers or logs in using JWT authentication.
3. The frontend sends HTTP requests to the Express REST APIs through Axios.
4. Express controllers process the requests.
5. Mongoose communicates with MongoDB to retrieve or update data.
6. Temple images are stored and served from the local uploads directory using Multer.
7. The backend returns JSON responses to the React frontend.
8. The frontend displays updated information to the user or administrator.

Technologies Used

- **Frontend:** React.js, HTML5, CSS3, JavaScript, Axios
- **Backend:** Node.js, Express.js
- **Authentication:** JWT, bcryptjs
- **Database:** MongoDB, Mongoose
- **File Upload:** Multer
- **Development Tools:** VS Code, Postman, Git, GitHub
- **Operating System:** Ubuntu Linux

Figure 1: Architecture and data flow of the voice patient diary sample application

Reference: <https://aws.amazon.com/blogs/industries/voice-applications-in-clinical-research-powered-by-ai-on-aws-part-1-architecture-and-design-considerations/>